

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
O QUE VOCÊ VIU NESTA AULA?	14
REFERÊNCIAS.....	15

EMBA

O QUE VEM POR AÍ?

Neste capítulo, exploraremos mais detalhadamente os métodos de aprendizagem por reforço baseados em valor e em política. Iremos analisar como o Q-learning, um dos algoritmos mais conhecidos, permite que agentes aprendam a tomar decisões ótimas através da atualização iterativa de uma função de valor de ação. Veremos a importância da Q-table e como a equação de Bellman é utilizada para ajustar os valores de Q com base nas recompensas recebidas e nas estimativas de valor futuro.

Além disso, discutiremos a estratégia epsilon-greedy, que equilibra a Exploration e a Exploitation durante o processo de aprendizagem, permitindo que o agente explore novas ações enquanto utiliza o conhecimento adquirido para maximizar recompensas. Entenderemos a importância dos parâmetros de taxa de aprendizagem (learning rate), fator de desconto (gamma) e a função de recompensa na calibração do algoritmo para diferentes problemas.

Por fim, abordaremos a evolução do Q-learning para o Deep Q-Network (DQN), que utiliza redes neurais profundas para lidar com espaços de estados de alta dimensionalidade. Veremos como o DQN se aplica a problemas complexos e discutiremos sua capacidade de aprender diretamente a partir de representações visuais do ambiente, como em jogos de vídeo. Este capítulo fornecerá uma visão abrangente das técnicas e algoritmos avançados de aprendizagem por reforço, preparando você para implementar soluções eficazes e inovadoras em diversos domínios.

HANDS ON

Nesse Hands On nós veremos uma simulação do treinamento do algoritmo Q-Learning sendo aplicado para um problema simplificado chamado Frozen Lake. Faremos essa simulação em um Notebook Python e o objetivo será analisarmos toda a execução do processo de aprendizagem, seus componentes e a criação da Q-table, como citado anteriormente.



SAIBA MAIS

Como dito anteriormente, na aprendizagem por reforço (RL) temos duas principais abordagens: métodos baseados em Valor e métodos baseados em Política. Os métodos baseados em valor são uma abordagem fundamental no RL, em que o objetivo é estimar e atualizar iterativamente o valor de cada estado ou ação com base nas recompensas recebidas. Existem várias técnicas para estimar os valores, sendo o Q-learning uma das mais proeminentes.

O Q-learning é um método de aprendizado por reforço que visa aprender a função de valor de ação ótima, representada por $Q(s, a)$, em que "s" é o estado atual e "a" é a ação a ser tomada.

A ideia central do Q-learning é iterativamente atualizar a função $Q(s, a)$ (Q-function) com base nas experiências do agente no ambiente. Isso é feito por meio de uma equação de atualização que considera a recompensa obtida, a estimativa do valor futuro e a taxa de aprendizado.

O processo de aprendizado do Q-learning é muitas vezes descrito como um jogo de busca de otimização, no qual o agente explora o ambiente, atualiza suas estimativas de $Q(s, a)$ e gradualmente converge para uma política ótima que maximiza a recompensa acumulada ao longo do tempo. Esse tal de $Q(s, a)$ pode ser descrito como uma medida de qualidade da tomada de uma determinada ação em um determinado estado. O resultado dessas atualizações é armazenado em uma estrutura chamada Q-table.

Vamos detalhar agora cada um dos componentes do algoritmo Q-learning e, em seguida, verificar a execução do algoritmo.

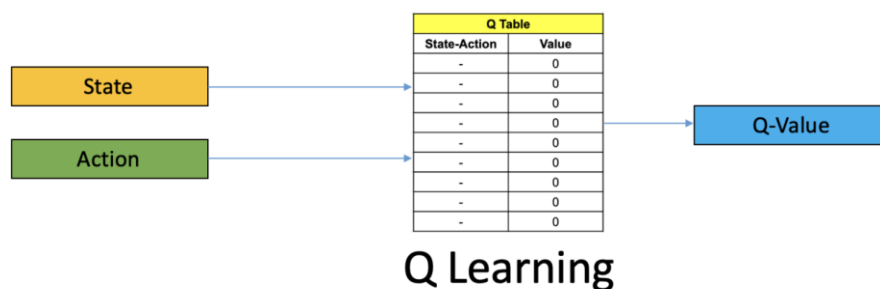


Figura 1 - Visão geral do algoritmo Q-Learning
Fonte: Choudhary (2023)

Vamos imaginar um exemplo simples em que temos um rato (agente) caminhando em um labirinto (ambiente), no qual o objetivo é coletar todo queijo (estados com queijo) do labirinto sem que ele caia na posição onde encontra-se o veneno (estado com veneno). Se o rato coletar a pilha maior de queijo, dizemos que o episódio termina com sucesso; se o rato passar pelo estado onde encontra-se o veneno, o episódio também termina, mas sem sucesso.

Esses dois estados são chamados de estados terminais. Podemos imaginar essa ideia de episódio como se fosse a "vida" de um jogo de videogame, em que tentamos passar de fase e às vezes conseguimos, às vezes acabamos "morrendo" na tentativa.

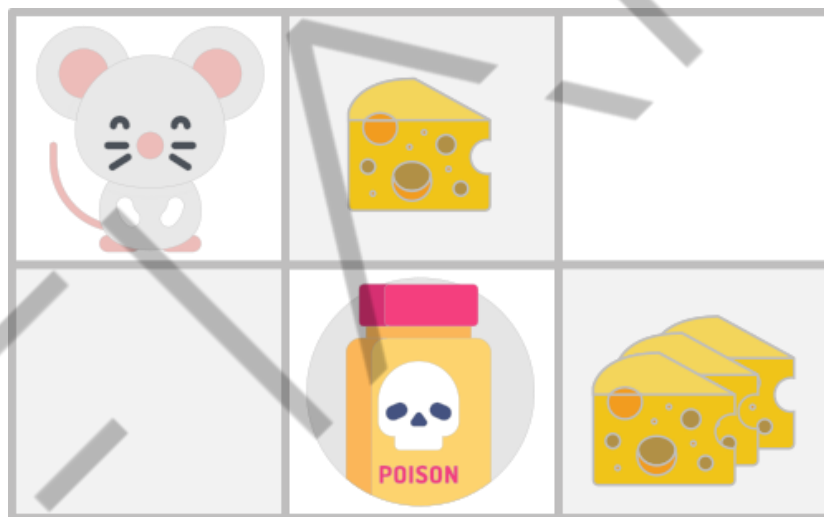
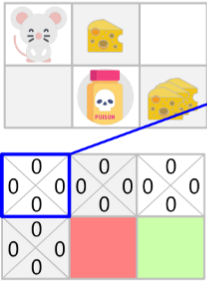


Figura 2 - Exemplo simplificado de algoritmo Q-Learning
Fonte: Huggingface (2024)

No **primeiro passo**, inicializamos uma Q-table vazia com todos os valores zerados; esta tabela contém, para cada estado e ação, os valores estado-ação correspondentes. Para este exemplo simples, o estado é definido apenas pela posição do rato. Portanto, temos 2 x 3 linhas na nossa Q-table, uma linha para cada posição possível do rato. Em cenários mais complexos, o estado pode conter mais informações do que apenas a posição do ator.

Nas colunas da Q-table temos as ações possíveis de serem realizadas no ambiente no qual o agente está inserido. Assim, temos a seguinte configuração de Q-table em que inicialmente todos os Q-values (células da tabela) estão zerados:



	Actions			
	←	→	↑	↓
Mouse at (1,1)	0	0	0	0
Mouse at (1,2)	0	0	0	0
Mouse at (1,3)	0	0	0	0
Mouse at (2,1)	0	0	0	0
Mouse at (2,2)	0	0	0	0
Mouse at (2,3)	0	0	0	0

Figura 3 - Exemplo de inicialização da Q-Table
Fonte: Hugginface (2024)

Como **segundo passo**, precisamos reduzir um parâmetro do algoritmo chamado épsilon (ϵ), que vai definir a probabilidade de fazermos exploration ou exploitation: quanto mais alto for o parâmetro épsilon mais exploration fazemos, quanto menor o parâmetro épsilon mais exploitation fazemos. No Q-Learning essa relação de ora fazer exploration, ora fazer exploitation é chamada de estratégia epsilon-greedy. Assim, vale a pena rever os conceitos da primeira relativa a exploration e exploitation.

Nos aprofundando matematicamente, a ideia é que com o valor inicial de $\epsilon = 1,0$:

- Com probabilidade $1 - \epsilon$: fazemos exploitation (nosso agente seleciona a ação com o maior valor do par estado-ação).
- Com probabilidade ϵ : fazemos exploration (tentando uma ação aleatória).

No início do treinamento, a probabilidade de fazermos exploration será enorme, já que ϵ é muito alto; então, na maioria das vezes, vamos explorar. Mas à medida que o treino avança e, conseqüentemente, nossa tabela Q fica cada vez melhor nas suas

estimativas, reduzimos progressivamente o valor ϵ , uma vez que precisaremos de cada vez menos exploration e mais exploitation.

Em um **terceiro passo** a ação (**a**) é tomada, seja via exploration ou exploitation. Recebemos a recompensa (**r**) dessa ação, bem como o novo estado (**s'**).

O **quarto e último passo** é a atualização da Q-table utilizando a equação de Bellman. Assim, temos o seguinte algoritmo que define todo o processo de aprendizado do Q-Learning:

- Por episódio no total de episódios de treino:
- Reduza o epsilon (já que precisamos cada vez menos de exploração).
- Redefinir o ambiente.
- Para passo em passos (até o máximo de passos):
 - Escolha a ação (**a**) pela estratégia epsilon-greedy (citada anteriormente).
 - Tome a ação (**a**) e observe o estado resultante (**s'**) e a recompensa (**r**).
 - Atualize o valor **Q-valor** de **Q(s,a)** usando a equação de Bellman:
 - $Q(s,a) + \alpha [R(s,a) + \max_{a'} Q(s',a') - Q(s,a)]$
 - Se terminar, termine o episódio.
 - Nosso próximo estado é o novo estado.

Mas o que é essa equação de Bellman? Ela é o coração dos métodos de valor no RL e permite que consigamos estimar a medida de qualidade da tomada de decisão de uma determinada ação em um determinado estado, que comentamos no início.

O trecho da equação **$\max Q(s',a')$** é o valor Q máximo do próximo estado **s'**, representando a recompensa futura esperada; em outras palavras, baseando-se nas experiencias anteriores, qual é a recompensa futura máxima possível considerando que tomei ação **a**.

Observando o exemplo do rato, imagine que após vários passos de execução do algoritmo já saberemos mais ou menos a recompensa futura de algumas ações em alguns estados pelo fato de o agente (rato) já ter tomado essas decisões, ou seja, baseado em experiencias anteriores.

Existem ainda alguns parâmetros que precisamos calibrar nesse algoritmo, que variam de problema para problema, mas que são essenciais:

- **Ir:** a taxa de aprendizagem (learning rate) no algoritmo Q-learning determina o quão rapidamente o agente aprende com novas informações. Ela controla o tamanho do passo que o agente dá ao atualizar os valores da tabela Q. Normalmente calibramos o Ir igual a 1.
- **Gamma:** é o fator de desconto (discount factor) e controla o peso dado às recompensas futuras. Ele determina o quão importante as recompensas futuras são em relação às recompensas imediatas, ajudando o agente a tomar decisões a longo prazo. Normalmente calibramos o gamma igual a 0.99.
- **Função de Recompensa R:** é a função que vai calcular a recompensa do agente ao tomar uma ação **a** em um determinado estado **s**. Pensando no exemplo do Rato, vamos imaginar uma função de recompensa da seguinte forma:
 - +0: Indo para um estado sem queijo.
 - +1: Indo para um estado com um pequeno queijo.
 - +10: Indo para o estado com a grande pilha de queijo.
 - -10: Indo para o estado com o veneno e, assim, morrendo.
 - +0: Se passarmos mais de cinco passos.
- **Quantidade de episódios:** a quantidade de episódios de treino define quantas vezes o agente interage com o ambiente para aprender a política ótima, que no nosso caso é calibrar a Q-table.
- **Quantidade máxima de passos:** define a quantidade de ações máxima por episódio. É um recurso que permite que o agente não fique preso eternamente em um único episódio caso exista alguma situação em que o agente não caia em nenhum estado circular. Por exemplo: o rato estar em um ambiente mais amplo e uma evolução incorreta do algoritmo fazer com que a rato ande em círculos eternamente.

Explorando um pouco mais a fundo a equação de Bellman, vamos retomar o exemplo do rato no labirinto. Já visualizamos o passo que é a inicialização da Q-table e o próximo é tomar uma ação pela estratégia epsilon-greedy. Por ser o início do algoritmo, a probabilidade de fazermos exploration (ação aleatória) é muito maior que exploitation. Vamos imaginar que a ação tomada foi caminhar para a direita:

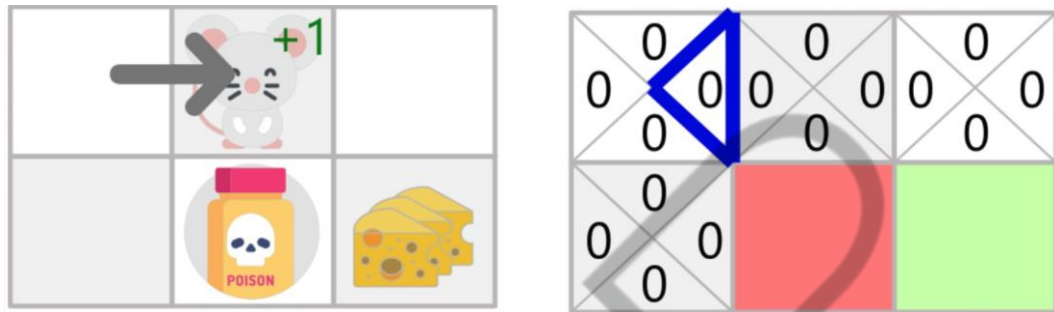


Figura 4 - Exemplo de tomada da ação no início da execução no Q-Learning
Fonte: Hugginface (2024)

A função de recompensa retornará **+1** e o rato estará agora em um novo estado não terminal e o episódio não é terminado. Agora temos todas as informações necessárias para atualizar o Q-value da célula correspondente a estado inicial e ação de ir para direita nesse estado. Assim, podemos substituir a nossa equação pelos seguintes valores:

$$Q(\text{estado_inicial}, \text{direita}) = Q(\text{estado_inicial}, \text{direita}) + \alpha [R(\text{estado_inicial}, \text{direita}) + \gamma \max_{a'} Q(\text{próximo_estado}, a') - Q(\text{estado_inicial}, \text{direita})]$$

$$Q(\text{estado_inicial}, \text{direita}) = 0 + 0.1 * [1 + 0.99 * 0 - 0]$$

$$Q(\text{estado_inicial}, \text{direita}) = 0.1$$

				
	0	0.1	0	0
	0	0	0	0
	0	0	0	0
	0	0	0	0
	0	0	0	0
	0	0	0	0

Figura 5 - Exemplo de atualização do Q-value no início da execução no Q-Learning
Fonte: Huggingface (2024)

Um passo importante a entendermos é que o trecho da equação de Bellman $\max Q(s',a')$ é igual a **0**, pois quando olhamos todos os Q-values no estado do queijo (estado s') todos estão zerados; do contrário, $\max Q(s',a')$ seria o maior valor desta linha da Q-table:

				
	0	0.1	0	0
	0	0	0	0
	0	0	0	0
	0	0	0	0
	0	0	0	0
	0	0	0	0

Figura 6 - Exemplo de atualização do Q-value no início da execução no Q-Learning com detalhamento do trecho $\max Q(s',a')$
Fonte: Adaptado Huggingface (2024)

Seguindo a execução do algoritmo, vamos imaginar que o agente tome a ação de ir para baixo (sempre baseado na estratégia epsilon-greedy) estando no estado 2, onde foi recebida a recompensa +1 do queijo.

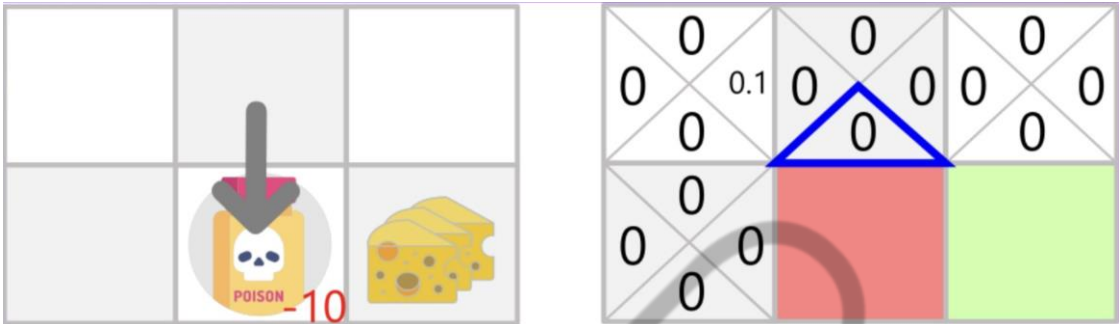


Figura 7 - Próximo passo da execução do algoritmo Q-Learning
Fonte: Adaptado Huggingface (2024)

Nesse cenário, a função de recompensa retornará **-10** e assim poderemos executar novamente a equação de Bellman para atualizar a Q-table, conforme podemos ver a seguir:

$$Q(\text{estado_2, baixo}) = 0 + 0.1 * [-10 + 0.99 * 0 - 0]$$
$$Q(\text{estado_2, baixo}) = -1$$

				
	0	0.1	0	0
	0	0	0	-1
	0	0	0	0
	0	0	0	0
	0	0	0	0
	0	0	0	0

Figura 8 - Execução do algoritmo Q-Learning no segundo passo
Fonte: Adaptado Huggingface (2024)

Dessa forma a Q-table vai sendo atualizada e, com a evolução dos episódios, o algoritmo vai deixando de fazer exploration (ação aleatória) e passa a fazer

exploitation na estratégia epsilon-greedy. No Q-learning fazer exploitation passa a ser tomar a ação de maior Q-value existente na Q-table em um determinado estado, reforçando ações de maior recompensa "histórica".

Embora o Q-learning seja um método de RL por valor, no fim temos a Q-table, que é uma representação de política.

Adicionalmente existe também o Deep Q-Network (DQN), que é uma extensão avançada do algoritmo Q-Learning projetada para lidar com ambientes mais complexos e de alta dimensionalidade.

A principal diferença entre o Q-Learning tradicional e o DQN é a maneira como a Q-Function é aproximada. Enquanto o Q-Learning usa uma Q-table para armazenar os Q-values para cada par estado-ação, o DQN utiliza uma rede neural profunda para estimar esses valores. Isso permite ao DQN lidar com espaços de estados grandes e contínuos, com alta dimensionalidade.

O processo de treinamento do DQN é semelhante ao do Q-Learning, mas em vez de atualizar uma tabela de valores Q a rede neural é treinada para minimizar a diferença entre os valores Q previstos e os valores Q observados durante a interação do agente com o ambiente. Isso é feito utilizando uma técnica chamada descida de gradiente.

Uma característica importante do DQN é a sua capacidade de lidar com problemas de aprendizado por reforço baseados em imagens, como jogos de vídeo. A rede neural recebe uma representação visual do estado do jogo como entrada e produz os Q-values para cada ação possível. Isso permite que o DQN aprenda diretamente a partir de pixels brutos, sem a necessidade de recursos de pré-processamento complexos.

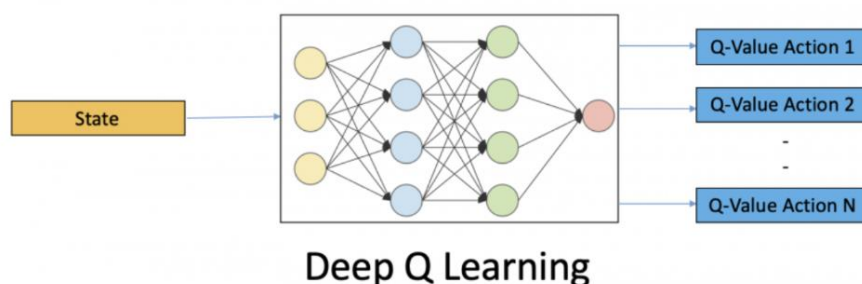


Figura 9 - Visão geral do algoritmo DQN
Fonte: Choudhary (2023)

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, exploramos o algoritmo Q-learning, uma técnica de Aprendizado por Reforço (RL) que visa aprender a função de valor de ação ótima para um agente em um ambiente. Inicialmente, entendemos que o RL pode ser abordado através de métodos baseados em Valor e em Política, com o Q-learning sendo um exemplo proeminente dos métodos baseados em Valor.

O Q-learning funciona atualizando iterativamente a função $Q(s, a)$ com base nas experiências do agente no ambiente. A função $Q(s, a)$ representa a qualidade de tomar uma determinada ação em um estado específico. Utiliza-se a equação de Bellman para realizar essas atualizações, levando em conta a recompensa imediata, a estimativa do valor futuro e a taxa de aprendizado.

No exemplo apresentado, um rato em um labirinto, cada estado é representado pela posição do rato e as ações possíveis são movimentos para cima, para baixo, para a esquerda ou para a direita. Inicialmente, a Q-table é inicializada com valores zerados. Durante a execução do algoritmo, o agente interage com o ambiente, tomando ações e recebendo recompensas, e a Q-table é atualizada de acordo.

O algoritmo segue uma estratégia epsilon-greedy para equilibrar o exploration e o exploitation, onde, inicialmente, a probabilidade de exploração é alta, mas diminui à medida que o treinamento avança. As atualizações na Q-table são realizadas de acordo com a equação de Bellman, utilizando parâmetros como taxa de aprendizagem e fator de desconto. A quantidade de episódios de treino e a quantidade máxima de passos são outros parâmetros importantes que influenciam o processo de aprendizado.

Ao longo do treinamento, a Q-table é refinada e o agente aprende a selecionar ações que levam a estados que oferecem recompensas mais altas. O Q-learning é um método poderoso e amplamente utilizado em RL para aprender políticas ótimas em ambientes complexos.

No fim, também foi citada a evolução do Q-Learning com a técnica Deep Q-Network, que substitui a Q-table por uma rede neural artificial, permitindo assim a resolução de problemas mais complexos.

REFERÊNCIAS

CHOUDHARY, A. **A Hands-On Introduction to Deep Q-Learning using OpenAI Gym in Python.** 2023. Disponível em: <<https://www.analyticsvidhya.com/blog/2019/04/introduction-deep-q-learning-python/>>. Acesso em: 18 jun. 2024.

GÉRON, A. **Hands-On Machine Learning with Scikit-Learn, Keras, and Tensorflow:** Concepts, Tools, and Techniques to Build Intelligent Systems. 3. ed. Sebastopol: O'Reilly Media, 2022.

HUGGING FACE. **Deep Reinforcement Learning Course.** 2024. Disponível em: <<https://huggingface.co/learn/deep-rl-course>>. Acesso em: 18 jun. 2024.

SUTTON, R. S.; BARTO, A. G. **Reinforcement Learning:** An Introduction. 2. ed. Cambridge: MIT Press, 2018.

PALAVRAS-CHAVE

Palavras-chave: Aprendizagem por Reforço. Aprendizagem de Máquina. Recompensa.

EMAP



POSTECH